TUTORIAL: Digital I/O

LED Toggle with Push-Button

Name: WonBitna ID: 22400462

I. Introduction

We will learn how to control digital I/O of GPIOs of the MCU board to turn on/off an LED with a push-button input. The LED should be turned on when the button is pressed.

The objectives of this lab are to learn how to

- Read and configure registers of digital GPIO of MCU
- Program firmware to control digital input/output pins
- Create your own functions for GPIOs

Hardware

NUCLEO -F411RE

Software

Keil uVision IDE, CMSIS, EC_HAL

Documentation

STM32 Reference Manual

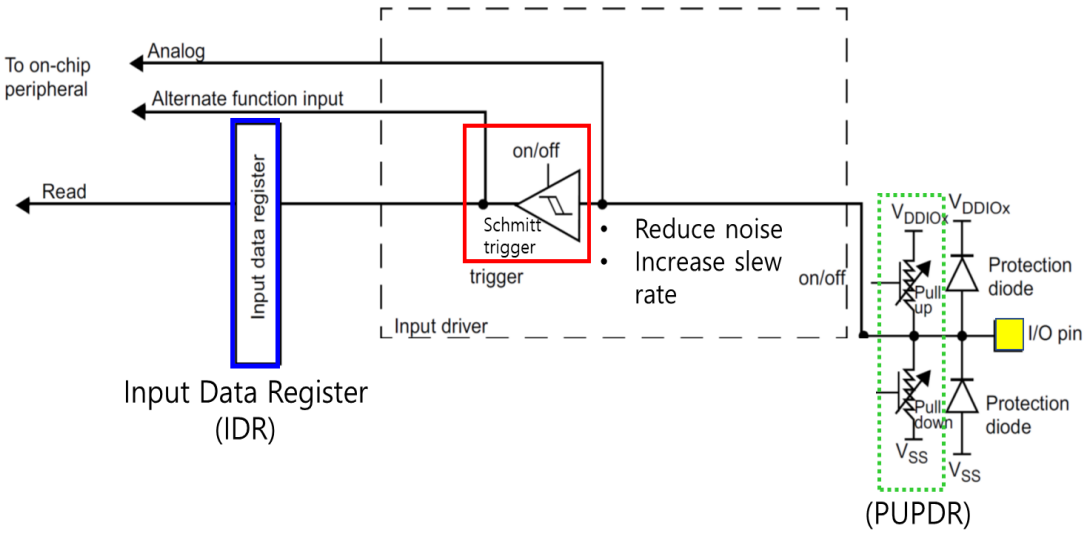
II. Basics of GPIO IN

A. GPIO Digital Out Register

List GPIO registers for this LAB

Type	Register Name	Description
GPIO	MODER	Mode: Input
	PUPDR	Pull-Up Pull-Down: No pull-up,down/Pull-up, Pull-down, Reserved
	IDR	Input Data Register

Schematic



Process of GPIOx register initiation

0. Enable Peripheral Clock (**AHB1ENR**)
1. Configure as Digital Input (**MODER**)
2. Configure pull-up/down resistors (**PUPDR**)
3. Read Data **(IDR)**

III. Tutorial

A. Register Configuration

1. GPIO: Digital In - Pin Initialization & Read PushButton

Port C Pin 13 / Input // Pull-Up

use **#define BUTTON_PIN 13**

- **MODER:** Input (MODER13[1:0]=[0 0])

```
GPIOC ->MODER &= ~(3<<( BUTTON_PIN *2)); // clear bits at both    [26] and [27]
```

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 2y:2y+1 **MODERy[1:0]:** Port x configuration bits (y = 0..15)
These bits are written by software to configure the I/O direction mode.
00: Input (reset state)
01: General purpose output mode
10: Alternate function mode
11: Analog mode

Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Initial	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
Logic	Bitwise AND																															
mask	1	1	1	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Value	x	x	x	x	0	0	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x

- **PUPDR:** pull-up (PUPDR13[1:0]=01)

```
GPIOC->PUPDR &= ~(3<<( BUTTON_PIN *2));
GPIOC->PUPDR | = (1<<BUTTON_PIN*2);
```

Paste Register map from reference manual

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 2y:2y+1 **PUPDRy[1:0]:** Port x configuration bits (y = 0..15)
These bits are written by software to configure the I/O pull-up or pull-down
00: No pull-up, pull-down
01: Pull-up
10: Pull-down
11: Reserved

- **IDR:** Read Push-Button Value (IDR13=0or1)

```
btVal = (GPIOC->IDR) & (1<<BUTTON_PIN);
```

Paste Register map from reference manual

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **IDRy**: Port input data (y = 0..15)
These bits are read-only and can be accessed in word mode only. They contain the input value of the corresponding I/O port.

B. Programming

Preparation

- Download ecRCC.h and ecRCC.c.
- Save them in the library folder: "**include\"**
  - You must have "ecPinNames.*, ecRCC.*" in include folder
- Create a new item(file) as '**\tutorial\TU_GPIO_Digital_InOut_Button'.c'**
- Copy and paste from the source code '**TU_GPIO_Digital_InOut_LED_Button_student.c**'.
- Create a new environment : '**TU_GPIO_Digital_InOut_Button**''.

  (platformio.ini) : `[env:TU_GPIO_Digital_InOut_Button]`

```
[env:TU_GPIO_Digital_InOut_Button]
build_src_filter = +<tutorial/TU_GPIO_Digital_InOut_Button_student.c>
+<include/*.c> -<include/ecSTM32_simple.c>
```

  - Here, we have removed ecSTM32_simple.h for it can cause redundant errors
  - You can also manually delete "ecSTM32_simple.*" files.

Exercise

Fill in the empty spaces in the code. Then, compile(F7) and flash(F8) the source code on the MCU board.

```
/*-----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : Won Bitna
Created          : 05-03-2021
Modified         : 09-13-2026
Language/ver     : C++ in VS Code

Description      : Tutorial Digital InOut
/-----------------------------------------------------------------*/



// GPIO Mode           : Input(00), Output(01), AlterFunc(10), Analog(11, reset)
// GPIO Speed          : Low speed (00), Medium speed (01), Fast speed (10), High
speed (11)
```

```
// GPIO Output Type     : Output push-pull (0, reset), Output open drain (1)
// GPIO Push-Pull       : No pull-up, pull-down (00), Pull-up (01), Pull-down
(10), Reserved (11)



#include "stm32f411xe.h"
#include "ecRCC2.h"

#define LED_PIN    5        //LD2
#define BUTTON_PIN 13       // B1 Button

int main(void) {
    /* Part 1. RCC GPIOA Register Setting */
        RCC_HSI_init();
        RCC_GPIOA_enable();
        RCC_GPIOC_enable();

    /*----------------------------------------------------------------
                  [EXERCISE]
    ---------------------------------------------------------------- */

    /// Part 2. GPIO Register Setting for OUTPUT ///
        // GPIO Mode Register MODE=OUTPUT
        GPIOA->MODER &=
~(3UL<<(LED_PIN*2));                                    // Clear '00' for Pin 5
        GPIOA->MODER |=
(1UL<<(LED_PIN*2));                                     // Set '01' for Pin 5



    /// Part 3. GPIO Register Setting for INPUT ///
        // GPIO Mode Register  MODE=INPUT
        GPIOC->MODER &=
~(3UL<<(BUTTON_PIN*2));                                   // 00:
Input

        // GPIO Pull-Up/Pull-Down Register
        GPIOC->PUPDR &= ~(3UL<<(BUTTON_PIN*2));
        GPIOC->PUPDR  |=
(2UL<<(BUTTON_PIN*2));                                    // 10: Pull-
down



    /* Button Value Initialization */
        unsigned int btVal=0;
```

```
    /* Part 4. loop  */
        while(1){
            //Read bit value of Button
            btVal= (GPIOC->IDR) & (1UL << BUTTON_PIN);
            if(btVal == 0)
                GPIOA->ODR |= (1UL << LED_PIN);
            else
                GPIOA->ODR &= ~(1UL << LED_PIN);
        }
}
```

Solution

```c
#define LED_PIN    5        //LD2
#define BUTTON_PIN 13       // B1 Button

int main(void) {
    /* Part 1. RCC GPIOA Register Setting */
        RCC_HSI_init();
        RCC_GPIOA_enable();
        RCC_GPIOC_enable();

    /* Part 2. GPIO Register Setting for OUTPUT*/
        // GPIO Mode Register
        GPIOA->MODER &= ~(3UL<<(2*LED_PIN)); // Clear '00' for Pin 5
        GPIOA->MODER |=   1UL<<(2*LED_PIN);  // Set '01' for Pin 5


    /* Part 3. GPIO Register Setting for INPUT*/
        // GPIO Mode Register
        GPIOC->MODER &= ~(3UL<<(2*BUTTON_PIN)); // 00: Input

        // GPIO Pull-Up/Pull-Down Register
        GPIOC->PUPDR &= ~(3UL<<(2*BUTTON_PIN)); // Clear '00'
        GPIOC->PUPDR  |= 2UL<<(2*BUTTON_PIN);   // 10: Pull-down

    /* Button Value Initialization */
        unsigned int btVal=0;

    /* Part 4. Dead loop  */
        while(1){
            //Read bit value of Button
            btVal=(GPIOC->IDR) & (1UL << BUTTON_PIN);
            if(btVal == 0)
                GPIOA->ODR |= (1UL << LED_PIN);
            else
                GPIOA->ODR &= ~(1UL << LED_PIN);
        }
}
```

Appendix

[See here for MCU resources]

1. Pin Configuration of NUCLE-F401RE

Figure 18. NUCLEO-F401RE

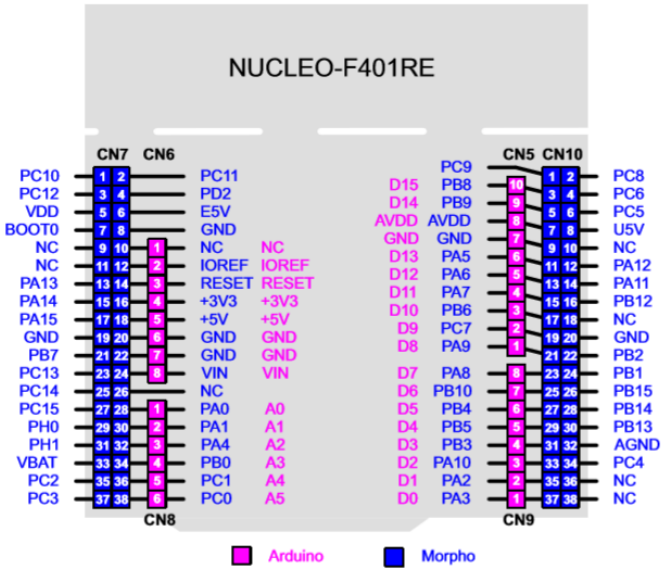
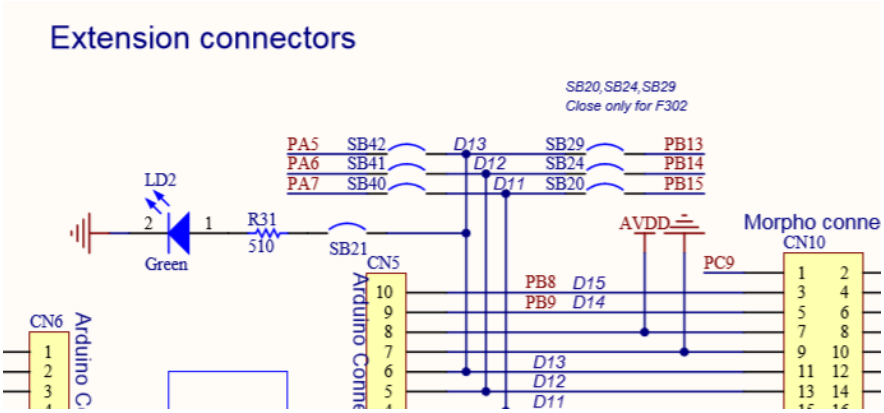


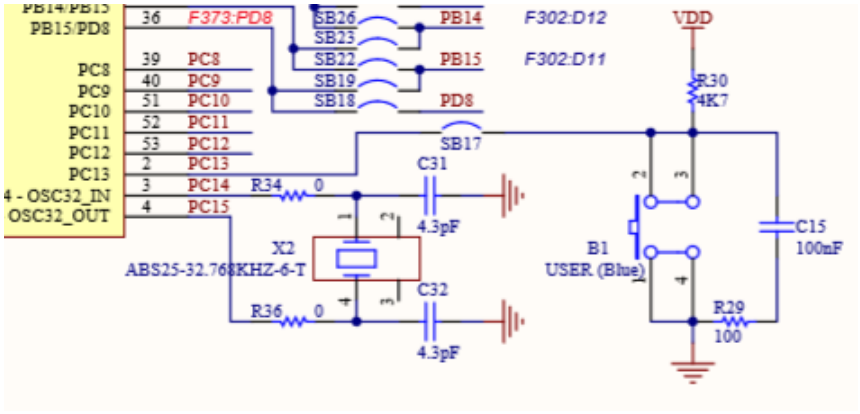
Table 29. ST morpho connector on NUCLEO-F401RE, NUCLEO-F411RE, NUCLEO-F446RE

CN7 odd pins		CN7 even pins		CN10 odd pins		CN10 even pins	
Pin	Name	Name	Pin	Pin	Name	Name	Pin
1	PC10	PC11	2	1	PC9	PC8	2
3	PC12	PD2	4	3	PB8	PC6	4
5	VDD	E5V	6	5	PB9	PC5	6
7	BOOT0(1)	GND	8	7	AVDD	U5V(2)	8
9	-	-	10	9	GND	-	10
11	-	IOREF	12	11	PA5	PA12	12
13	PA13(3)	RESET	14	13	PA6	PA11	14
15	PA14(3)	+3.3V	16	15	PA7	PB12	16
17	PA15	+5V	18	17	PB6	-	18
19	GND	GND	20	19	PC7	GND	20
21	PB7	GND	22	21	PA9	PB2	22
23	PC13	VIN	24	23	PA8	PB1	24
25	PC14	-	26	25	PB10	PB15	26
27	PC15	PA0	28	27	PB4	PB14	28
29	PH0	PA1	30	29	PB5	PB13	30
31	PH1	PA4	32	31	PB3	AGND	32
33	VBAT	PB0	34	33	PA10	PC4	34
35	PC2	PC1 or PB9(4)	36	35	PA2	-	36
37	PC3	PC0 or PB8(4)	38	37	PA3	-	38

1. Default state of BOOT0 is 0. It can be set to 1 when a jumper is on pin5-7 of CN7.Two unused jumpers are available on CN11 and CN12 (bottom side of the board).
2. U5V is 5 V power from ST-LINK/V2-1 USB connector and it rises before +5V.
3. PA13 and PA14 share with SWD signals connected to ST-LINK/V2-1, it is not recommend to use them as IO pins if ST-LINK part is not cut.
4. Refer to *Table 10: Solder bridges* for details.

2. LED/Button Circuit Diagram





이거 지터 방지를 위해 소프트웨어/하드웨어적으로 어떤걸 할수있냐... 이런게 시험인가 퀴즈에 나왔었다...